UNIVERSIDAD AUTÓNOMA DE SAN LUIS POTOSÍ

FACULTAD DE INGENIERÍA

COMPUTO EN LA NUBE
CHAT MULTIUSUARIO



UNIVERSIDAD AUTÓNOMA
DE SAN LUIS POTOSÍ

JACOBO RODRIGUEZ TORRES

ALBERTO OTONIEL LOPEZ ROMERO

JOSE ORLANDO JUAREZ RAMIREZ

ELOY EMILIO HERNANDEZ HERNANDEZ

KARLA DAYANA MEJIA PADRON

FRANCISCO ABUNDIS AVALOS

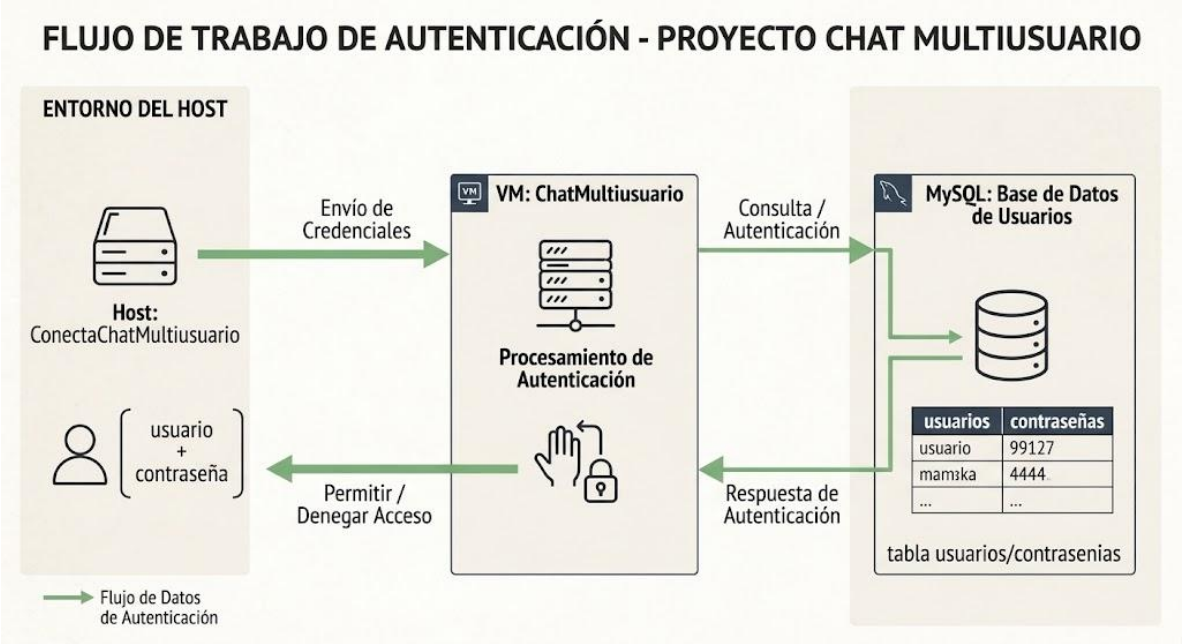
1. Introducción

En esta práctica se ampliará el funcionamiento del servicio de chat multiusuario desarrollado anteriormente. La versión original permite que diferentes clientes se conecten a un servidor mediante sockets e intercambien mensajes en tiempo real.

La nueva versión incorporará un mecanismo de autenticación. Antes de acceder al chat, cada cliente deberá proporcionar un nombre de usuario y una contraseña. El servidor recibirá las credenciales y consultará una base de datos MySQL para comprobar que correspondan a un usuario registrado y activo.

Se utilizará nuevamente el modelo cliente-servidor:

- Los clientes solicitan acceso al chat y envían sus credenciales.
- El servidor recibe las credenciales y realiza la validación.
- MySQL almacena la información de los usuarios autorizados.
- Solo los clientes autenticados pueden entrar al chat y enviar mensajes.



2. Objetivo

Implementar un mecanismo de autenticación para el chat multiusuario, utilizando una base de datos MySQL en la que se almacenarán los nombres de usuario y contraseñas de las personas autorizadas.

El sistema deberá:

- Solicitar un nombre de usuario y una contraseña.
- Enviar las credenciales al servidor.
- Consultar la base de datos desde el servidor.
- Permitir el acceso cuando las credenciales sean correctas.
- Rechazar la conexión cuando sean incorrectas.
- Permitir la comunicación únicamente entre usuarios autenticados.

3. Arquitectura del sistema

El proyecto se divide en dos aplicaciones.

Proyecto servidor: ChatMultiusuario

Se ejecuta en la máquina virtual y contiene las clases:

- ChatMultiusuario
- Servidor
- ManejadorClientes
- ConectarMySQL
- UsuarioDAO

También se encuentra en la máquina virtual el servidor MySQL con la base de datos restordb.

Proyecto cliente: ConectaChatMultiusuario

Se ejecuta en la máquina host y contiene:

- ConectaChatMultiusuario
- Cliente

El cliente no se conecta directamente a MySQL. Las credenciales se envían mediante el socket al servidor y este se encarga de realizar la consulta.

4. Creación de la tabla de usuarios

La base de datos utilizada se llama restordb. Dentro de ella se creará una tabla para guardar los usuarios autorizados.

```
USE restordb;

CREATE TABLE usuarios (
    id_usuario INT AUTO_INCREMENT PRIMARY KEY,
    nombre_usuario VARCHAR(50) NOT NULL UNIQUE,
    contrasena VARCHAR(100) NOT NULL,
    nombre_completo VARCHAR(100),
    activo BOOLEAN NOT NULL DEFAULT TRUE,
    fecha_registro TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

INSERT INTO usuarios
    (nombre_usuario, contrasena, nombre_completo)
VALUES
    ('otto', '12345', 'Otoniel López'),
    ('ana', 'ana123', 'Ana Martínez'),
    ('carlos', 'carlos123', 'Carlos Hernández');

SELECT * FROM usuarios;
```

Modificaciones del servidor

5. Clase ConectarMySQL

En el proyecto ChatMultiusuario se crea la clase ConectarMySQL. Esta clase tiene la responsabilidad de abrir la conexión con la base de datos.

```
package Main;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public class ConectarMySQL {

    private static final String URL =
            "jdbc:mysql://localhost:3306/restordb"
            + "?useSSL=false"
            + "&allowPublicKeyRetrieval=true"
            + "&serverTimezone=UTC";

    private static final String USER = "USUARIO_MYSQL";
    private static final String PASSWORD = "CONTRASENA_MYSQL";

    public Connection getConnection() throws SQLException {
        return DriverManager.getConnection(
                URL,
                USER,
                PASSWORD
        );
    }
}
```

Se utiliza localhost porque el servidor del chat y MySQL se ejecutan dentro de la misma máquina virtual.

En USER y PASSWORD deberán colocarse las credenciales reales utilizadas para ingresar a MySQL.

Estas credenciales no son las mismas que utilizarán otto, ana o carlos para entrar al chat. Su única función es permitir que Java se conecte a MySQL.

6. Clase UsuarioDAO

Después se crea la clase UsuarioDAO. Una clase DAO, del inglés Data Access Object, se encarga de realizar operaciones relacionadas con la base de datos.

En este proyecto su función será buscar un usuario con el nombre y la contraseña recibidos.

```
package Main;

import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;

public class UsuarioDAO {

    public boolean autenticar(
            String usuario,
            String contrasena
    ) {
        String consulta = """
                SELECT id_usuario
                FROM usuarios
                WHERE nombre_usuario = ?
                AND contrasena = ?
                AND activo = TRUE
                """;
        ConectarMySQL conexionBD = new ConectarMySQL();

        try (
            Connection conexion =
                    conexionBD.getConnection();
            PreparedStatement sentencia =
                    conexion.prepareStatement(consulta)
        ) {
            sentencia.setString(1, usuario);
            sentencia.setString(2, contrasena);

            try (ResultSet resultado =
                         sentencia.executeQuery()) {
                return resultado.next();
            }
        } catch (SQLException e) {
            System.out.println(
                    "Error al autenticar: "
                    + e.getMessage()
            );
            return false;
        }
    }
}
```

7. Modificación de ManejadorClientes

La clase ManejadorClientes debe recibir las credenciales antes de incorporar al cliente al arreglo de usuarios conectados.

```
package Main;

import java.io.BufferedReader;
import java.io.BufferedWriter;
import java.io.IOException;
import java.io.InputStreamReader;
import java.io.OutputStreamWriter;
import java.net.Socket;
import java.util.ArrayList;

public class ManejadorClientes implements Runnable {
    public static ArrayList<ManejadorClientes> arregloClientes =
            new ArrayList<>();

    private Socket socket;
    private BufferedReader recibeMsg;
    private BufferedWriter enviaMsg;
    private String usuario;
    private boolean autenticado;

    public ManejadorClientes(Socket socket) {
        try {
            this.socket = socket;
            this.enviaMsg = new BufferedWriter(
                    new OutputStreamWriter(socket.getOutputStream())
            );
            this.recibeMsg = new BufferedReader(
                    new InputStreamReader(socket.getInputStream())
            );
            this.usuario = recibeMsg.readLine();
            String contrasena = recibeMsg.readLine();
            UsuarioDAO usuarioDAO = new UsuarioDAO();
            autenticado = usuarioDAO.autenticar(usuario, contrasena);
            if (autenticado) {
                enviaMsg.write("AUTH_OK");
                enviaMsg.newLine();
                enviaMsg.flush();
                synchronized (arregloClientes) {
                    arregloClientes.add(this);
                }
                System.out.println(
                        "Usuario autenticado: " + usuario
                );
                broadcastMensaje(
                        "SERVIDOR: " + usuario + " ha entrado al chat!"
                );
            } else {
                enviaMsg.write("AUTH_ERROR");
                enviaMsg.newLine();
                enviaMsg.flush();

                System.out.println(
                        "Intento de acceso rechazado para: " + usuario
                );
            }
        } catch (IOException e) {
            autenticado = false;
            System.out.println(
                    "Error al recibir las credenciales: " + e.getMessage()
            );
        }
    }
    public boolean estaAutenticado() {
        return autenticado;
    }
```

```
    @Override
    public void run() {
        if (!autenticado) {
            cierraConexion();
            return;
        }
        while (socket != null && !socket.isClosed()) {
            try {
                String mensajeCliente = recibeMsg.readLine();

                if (mensajeCliente == null) {
                    break;
                }
                broadcastMensaje(mensajeCliente);
            } catch (IOException e) {
                break;
            }
        }
        cierraTodo();
    }
    public void broadcastMensaje(String mensaje) {
        synchronized (arregloClientes) {
            for (ManejadorClientes cliente : arregloClientes) {
                try {
                    if (cliente != this) {
                        cliente.enviaMsg.write(mensaje);
                        cliente.enviaMsg.newLine();
                        cliente.enviaMsg.flush();
                    }
                } catch (IOException e) {
                    System.out.println(
                            "No se pudo enviar un mensaje a "
                            + cliente.usuario
                    );
                }
            }
        }
    }
    public void eliminaManejadorCliente() {
        boolean eliminado;

        synchronized (arregloClientes) {
            eliminado = arregloClientes.remove(this);
        }

        if (eliminado) {
            broadcastMensaje(
                    "SERVIDOR: " + usuario + " ha abandonado el chat!"
            );
            System.out.println(
                    "El usuario " + usuario + " se desconectó."
            );
        }
    }
    public void cierraTodo() {
        eliminaManejadorCliente();
        cierraConexion();
    }
    private void cierraConexion() {
        try {
            if (recibeMsg != null) {
                recibeMsg.close();
            }
            if (enviaMsg != null) {
                enviaMsg.close();
            }
            if (socket != null && !socket.isClosed()) {
                socket.close();
            }
        } catch (IOException e) {
            System.out.println(
                    "Error al cerrar la conexión: " + e.getMessage()
            );
        }
    }
}
```

8. Modificación de Servidor

En la clase Servidor, el método iniciaServidor() debe comprobar el estado de la autenticación antes de crear el hilo.

```
package Main;

import java.io.IOException;
import java.net.ServerSocket;
import java.net.Socket;

public class Servidor {

    private ServerSocket socketServidor;

    public Servidor(ServerSocket socketServidor) {
        this.socketServidor = socketServidor;
    }

    public void iniciaServidor() {
        try {
            while (!socketServidor.isClosed()) {
                Socket socket = socketServidor.accept();

                System.out.println(
                        "Un cliente intenta conectarse..."
                );

                ManejadorClientes manejador =
                        new ManejadorClientes(socket);

                if (manejador.estaAutenticado()) {
                    Thread hilo = new Thread(manejador);
                    hilo.start();
                }
            }

        } catch (IOException e) {
            cierraSocketServidor();
        }
    }

    public void cierraSocketServidor() {
        try {
            if (socketServidor != null
                    && !socketServidor.isClosed()) {
                socketServidor.close();
            }

        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

Modificaciones del cliente

9. Modificación de ConectaChatMultiusuario

La clase principal del cliente ahora solicitará dos datos:

- Nombre de usuario.
- Contraseña.

```
package Main;

import java.io.IOException;
import java.net.Socket;
import java.util.Scanner;

public class ConectaChatMultiusuario {

    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);

        try {
            System.out.println("Usuario:");
            String usuario = input.nextLine();

            System.out.println("Contraseña:");
            String contrasena = input.nextLine();

            // Cambia esta dirección por la IP real de tu VM.
            Socket socket = new Socket("192.168.1.85", 5555);

            Cliente cliente = new Cliente(
                    socket,
                    usuario,
                    contrasena
            );

            if (cliente.autenticar()) {
                System.out.println(
                        "Autenticación correcta. Entraste al chat."
                );

                cliente.recibeMensaje();
                cliente.enviaMensaje();

            } else {
                System.out.println(
                        "Usuario o contraseña incorrectos."
                );

                cliente.cierraTodo();
            }

        } catch (IOException e) {
            System.out.println(
                    "No fue posible conectarse al servidor."
            );
            e.printStackTrace();
        }
    }
}
```

10. Modificación de Cliente

La clase Cliente ahora debe guardar la contraseña:

```
package Main;

import java.io.BufferedReader;
import java.io.BufferedWriter;
import java.io.IOException;
import java.io.InputStreamReader;
import java.io.OutputStreamWriter;
import java.net.Socket;
import java.util.Scanner;

public class Cliente {

    private Socket socket;
    private BufferedReader recibeMsg;
    private BufferedWriter enviaMsg;

    private String usuario;
    private String contrasena;

    public Cliente(
            Socket socket,
            String usuario,
            String contrasena
    ) throws IOException {

        this.socket = socket;
        this.usuario = usuario;
        this.contrasena = contrasena;

        this.recibeMsg = new BufferedReader(
                new InputStreamReader(socket.getInputStream())
        );

        this.enviaMsg = new BufferedWriter(
                new OutputStreamWriter(socket.getOutputStream())
        );
    }

    public boolean autenticar() {
        try {
            // Primero se envían las credenciales.
            enviaMsg.write(usuario);
            enviaMsg.newLine();

            enviaMsg.write(contrasena);
            enviaMsg.newLine();

            enviaMsg.flush();

            // Esperamos la respuesta del servidor antes
            // de comenzar a enviar mensajes.
            String respuesta = recibeMsg.readLine();

            return "AUTH_OK".equals(respuesta);

        } catch (IOException e) {
            System.out.println(
                    "Error durante la autenticación: "
                    + e.getMessage()
            );
            return false;
        }
    }
```

```
    public void enviaMensaje() {

        Scanner teclado = new Scanner(System.in);

        try {
            while (socket.isConnected()
                    && !socket.isClosed()) {

                String mensaje = teclado.nextLine();

                enviaMsg.write(
                        usuario + ": " + mensaje
                );

                enviaMsg.newLine();
                enviaMsg.flush();
            }

        } catch (IOException e) {
            cierraTodo();
        }
    }

    public void recibeMensaje() {
        Thread hiloRecepcion = new Thread(() -> {
            try {
                while (socket.isConnected()
                        && !socket.isClosed()) {

                    String mensaje = recibeMsg.readLine();

                    if (mensaje == null) {
                        break;
                    }

                    System.out.println(mensaje);
                }

            } catch (IOException e) {
                System.out.println(
                        "Se perdió la conexión con el servidor."
                );
            } finally {
                cierraTodo();
            }
        });

        hiloRecepcion.start();
    }

    public void cierraTodo() {
        try {
            if (recibeMsg != null) {
                recibeMsg.close();
            }

            if (enviaMsg != null) {
                enviaMsg.close();
            }

            if (socket != null && !socket.isClosed()) {
                socket.close();
            }

        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

Ejecución y pruebas

12. Orden de ejecución

Para probar el sistema se utiliza el siguiente orden:

1. Iniciar el servicio de MySQL en la máquina virtual.
2. Comprobar que existe la base de datos restordb.
3. Comprobar que la tabla usuarios contiene registros.
4. Ejecutar la clase ChatMultiusuario en la VM.
5. Ejecutar la clase ConectaChatMultiusuario en el host.
6. Proporcionar un usuario y una contraseña registrados.
7. Abrir una segunda instancia del cliente.
8. Autenticar otro usuario.
9. Intercambiar mensajes entre los clientes.

13. Prueba con credenciales correctas

Al iniciar el servidor deberá aparecer:

```
Servidor iniciado en el puerto 5555...
```

Cuando un usuario proporciona credenciales correctas:

```
Un cliente intenta conectarse...
Usuario autenticado: otto
```

En el cliente se mostrará:

```
Usuario:
otto
Contraseña:
12345
Autenticación correcta.
```

14. Prueba con credenciales incorrectas

Si la contraseña no coincide con el registro de MySQL, el servidor mostrará:

Un cliente intenta conectarse...

Intento de acceso rechazado para: otto

En el cliente aparecerá:

Usuario o contraseña incorrectos.

El cliente rechazado no será agregado al arreglo de clientes y no podrá enviar ni recibir mensajes.